Diseño de agentes: patrones de planificación y memoria

Teoría de la masterclass · Una historia inspirada en Memento

Elena Díaz (@HelenDiMo)

1. La idea en una frase

Un agente es **un LLM que funciona en bucle**: decide qué hacer, usa herramientas, mira el resultado y vuelve a decidir. En cada vuelta tiene que responder dos preguntas:

- **¿Qué hago ahora?** → *planificación*
- **¿Qué ha pasado hasta ahora?** → *memoria*

En la película *Memento* (2000), Leonard no puede crear recuerdos nuevos. Para investigar usa un **método** (planificación) y unas **notas, polaroids y tatuajes** (memoria). Un LLM está en la misma situación.

En la película	En el agente	Para qué sirve
Leonard	El LLM	Razona y decide, pero no recuerda
El archivo policial, su cámara	Herramientas	Actuar y conseguir datos
Su método para investigar	Planificación	Decidir el siguiente paso o el plan entero
Polaroids y tatuajes	Memoria	No empezar de cero

2. Por qué un LLM es Leonard

¿Qué es? Un LLM es *stateless* (sin estado): cada llamada empieza en blanco.

¿Por qué importa? Lo que parece «memoria» en un chat es que la aplicación **reenvía todo el historial** en cada llamada. Ese historial tiene que caber en la **ventana de contexto**, que es limitada. Lo que no cabe, se olvida.

¿Cómo se usa? Para que un agente recuerde, hay que construirle la memoria fuera del modelo (secciones 6 y 7).

3. Herramientas

¿Qué es? Una herramienta es una función de Python con una descripción (*docstring*). El LLM lee la descripción para decidir cuándo usarla.

¿Por qué importa? El LLM **no ejecuta** la herramienta: solo devuelve qué herramienta quiere usar y con qué argumentos. Alguien tiene que ejecutarla y devolverle el resultado. Ese alguien es el bucle del agente.

¿Cómo se usa?

```
@tool
def buscar_archivo(tema: str) -> str:
    """Busca en el archivo policial. Temas: informe, motel, coche,
taller."""
    ...

llm_con_manos = llm.bind_tools([buscar_archivo])
```

4. Patrones de planificación

4.1 ReAct (Razonar + Actuar)

¿Qué es? El agente alterna **pensamiento → acción → observación** y repite hasta poder responder. No hay plan escrito: decide solo el siguiente paso.

¿Por qué importa? Es el patrón base de casi todos los agentes. Se adapta bien a las sorpresas porque cada paso se decide con lo último que ha visto.

¿Cómo se usa? En LangGraph, un grafo con dos nodos y un bucle:

- nodo `agente`: llama al LLM con las herramientas;
- nodo `herramientas` (`ToolNode`): ejecuta lo que pidió el LLM;
- arista condicional (`tools_condition`): si el LLM pidió herramientas, va a `herramientas`; si no, termina.

Riesgo: puede dar vueltas en bucle sin avanzar.

Origen: Yao et al. (2022), «ReAct: Synergizing Reasoning and Acting in Language Models».

4.2 Plan-and-Execute

¿Qué es? Primero un paso del LLM escribe **el plan completo**; después se ejecuta cada paso en orden.

¿Por qué importa? Es más ordenado y fácil de seguir. Si un paso falla, se puede añadir un paso de **replanificación** que reescriba el plan con lo aprendido.

¿Cómo se usa? El plan sale del LLM como un modelo de Pydantic (`llm.with_structured_output(Plan)`), y un grafo recorre los pasos.

Riesgo: si algo cambia y no hay replanificación, el plan se vuelve rígido.

	ReAct	Plan-and-Execute
El plan	Implícito, paso a paso	Explícito, al principio

	ReAct	Plan-and-Execute
Ventaja	Se adapta a sorpresas	Ordenado y fácil de seguir
Riesgo	Dar vueltas en bucle	Plan rígido si algo cambia

4.3 Reflexión

¿Qué es? Antes de entregar la respuesta, otro paso del LLM (o otro LLM) la **critica**. Si no la aprueba, el agente lo intenta otra vez con la crítica.

¿Por qué importa? Detecta datos inventados o contradicciones con las pistas.

¿Cómo se usa? Un nodo `revisor` que devuelve un veredicto estructurado (`aprobado`, `critica`) y una arista que vuelve al agente si no está aprobado. Siempre con un **máximo de intentos**, porque cada revisión es otra llamada al LLM.

Origen: Shinn et al. (2023), «Reflexion: Language Agents with Verbal Reinforcement Learning».

4.4 Para saber más

Hay variantes más avanzadas, como **ReWOO** (planifica todas las llamadas a herramientas de golpe) o **Tree of Thoughts** (explora varios caminos de razonamiento). No las vemos en esta sesión.

5. Tipos de memoria

La clasificación viene del marco **CoALA** (Sumers et al., 2023), el mismo que usa la documentación de LangGraph.

Tipo	En la película	En un agente
Corto plazo	Lo que Leonard tiene en la cabeza ahora	El historial de la conversación actual
Largo plazo: semántica	Los tatuajes: hechos	Datos sobre el usuario o el mundo
Largo plazo: episódica	Las polaroids: momentos vividos	Ejemplos de tareas pasadas
Largo plazo: procedimental	Su rutina: mirar siempre sus notas	Las instrucciones del prompt

6. Memoria a corto plazo en LangGraph: el checkpointer

¿Qué es? Guarda el estado del grafo (los mensajes) de cada **hilo**, identificado por `thread_id`.

¿Por qué importa? Mismo hilo: el agente recuerda la conversación. Hilo nuevo: empieza en blanco, como Leonard al despertar.

¿Cómo se usa?

```
from langgraph.checkpoint.memory import InMemorySaver

agente = grafo.compile(checkpointer=InMemorySaver())
config = {"configurable": {"thread_id": "dia-1"}}
agente.invoke({"messages": [("user", "Hola")]}, config)
```

Problema: el historial crece en cada turno. Soluciones habituales: recortar los mensajes antiguos o resumirlos.

7. Memoria a largo plazo en LangGraph: el Store

¿Qué es? Un almacén clave-valor organizado en *namespaces* (como carpetas) que **comparten todos los hilos**.

¿Por qué importa? Es lo que permite que el agente recuerde hechos aunque empiece una conversación nueva.

¿Cómo se usa?

```
from langgraph.store.memory import InMemoryStore
from langgraph.config import get_store

@tool
def tatuar(hecho: str, fuente: str) -> str:
    """Guarda un hecho importante para siempre."""
    get_store().put(("leonard", "tatuajes"), str(uuid.uuid4()),
                    {"hecho": hecho, "fuente": fuente})
    return "Tatuado"

agente = grafo.compile(checkpointer=InMemorySaver(),
store=InMemoryStore())
```

`InMemorySaver` e `InMemoryStore` se borran al reiniciar el kernel. En producción se usan versiones con base de datos.

8. Memoria envenenada

¿Qué es? Un dato falso, viejo o manipulado que entra en la memoria del agente.

¿Por qué importa? El agente confía en su memoria igual que en un hecho comprobado, y repite el dato falso con total seguridad. Igual que Leonard con sus notas.

¿Cómo se defiende?

- **Guardar la fuente** de cada recuerdo.
- **Validar al escribir:** no todo merece guardarse.
- **Comprobar antes de usar** lo que no tiene fuente fiable.
- **Caducidad:** un dato antiguo puede estar desactualizado.

Una regla en el prompt ayuda, pero no es una garantía: el LLM puede ignorarla.

9. Resumen

1. Un agente es un LLM en bucle con herramientas. El LLM, por sí solo, no recuerda nada.
2. ReAct decide paso a paso; Plan-and-Execute escribe el plan antes; Reflexión revisa antes de entregar.
3. Corto plazo = el hilo (checkpointer). Largo plazo = entre hilos (Store).
4. La memoria también miente: guarda la fuente y comprueba antes de confiar.

Referencias

- Yao, S. et al. (2022). *ReAct: Synergizing Reasoning and Acting in Language Models*. arXiv:2210.03629.
- Shinn, N. et al. (2023). *Reflexion: Language Agents with Verbal Reinforcement Learning*. arXiv:2303.11366.
- Sumers, T. et al. (2023). *Cognitive Architectures for Language Agents (CoALA)*. arXiv:2309.02427.
- Documentación de LangGraph: memoria y agentes (docs.langchain.com).

Código probado con LangGraph 1.2. La API de LangGraph cambia entre versiones: si algo no funciona, revisa la versión instalada.